

Index

NO	Experiment Name
1	Design a calculator with basic mathematic operation using GUI component.
2	Write a Java program to convert binary to decimal and decimal to binary using GUI component./Temperature converter
3	Design a 2Player Tic-tac-toe using Java GUI.
4	Write a Java program to illustrate suspend, resume and stop operation of thread.
5	Write a Java program to create thread class.
6	Write a Java program to illustrate yield(), stop() and sleep() method using thread.
7	Write a client and server program in Java to establish a connection between them.
8	Design a form with Java GUI to perform insert, delete, edit and show data using Excel database.

Code:

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {

    private final JTextField display = new JTextField("0");

    private double num = 0;

    private String op = "";

    private boolean newInput = true;

    public Calculator() {

        setTitle("Calculator");

        setSize(300, 400);

        setDefaultCloseOperation(EXIT_ON_CLOSE);

        setLayout(new BorderLayout());

        display.setFont(new Font("Arial", Font.PLAIN, 24));

        display.setHorizontalAlignment(JTextField.RIGHT);

        display.setEditable(false);

        add(display, BorderLayout.NORTH);

        JPanel buttons = new JPanel(new GridLayout(4, 4, 5, 5));

        String[][] btnLabels = {

            { "7", "8", "9", "/" },

            { "4", "5", "6", "*" },

            { "1", "2", "3", "-" },
```

```

        { "0", "C", "=", "+" }
    };

    for (String[] row : btnLabels) {
        for (String label : row) {
            JButton btn = new JButton(label);
            btn.addActionListener(this);
            buttons.add(btn);
        }
    }
    add(buttons, BorderLayout.CENTER);
}

public void actionPerformed(ActionEvent e) {
    String cmd = e.getActionCommand();
    if (cmd.matches("[0-9]")) {
        if (newInput) {
            display.setText("");
            newInput = false;
        }
        display.setText(display.getText() + cmd);
    } else if (cmd.equals(".")) {
        if (newInput)
            display.setText("0.");
        else if (!display.getText().contains("."))
            display.setText(display.getText() + ".");
        newInput = false;
    } else if (cmd.matches("[+\\-*/]")) {

```

```

    if (!op.isEmpty() && !newInput)
        calculate();
    num = Double.parseDouble(display.getText());
    op = cmd;
    newInput = true;
} else if (cmd.equals("=") && !op.isEmpty()) {
    calculate();
    op = "";
} else if (cmd.equals("C")) {
    display.setText("0");
    num = 0;
    op = "";
    newInput = true;
}
}

```

```

private void calculate() {
    double val = Double.parseDouble(display.getText());
    double result = switch (op) {
        case "+" -> num + val;
        case "-" -> num - val;
        case "*" -> num * val;
        case "/" -> val != 0 ? num / val : Double.NaN;
        default -> val;
    };
    display.setText(Double.isNaN(result) ? "Error"
        : result == (long) result ? String.valueOf((long) result) : String.valueOf(result));
    newInput = true;
}

```

```
}
```

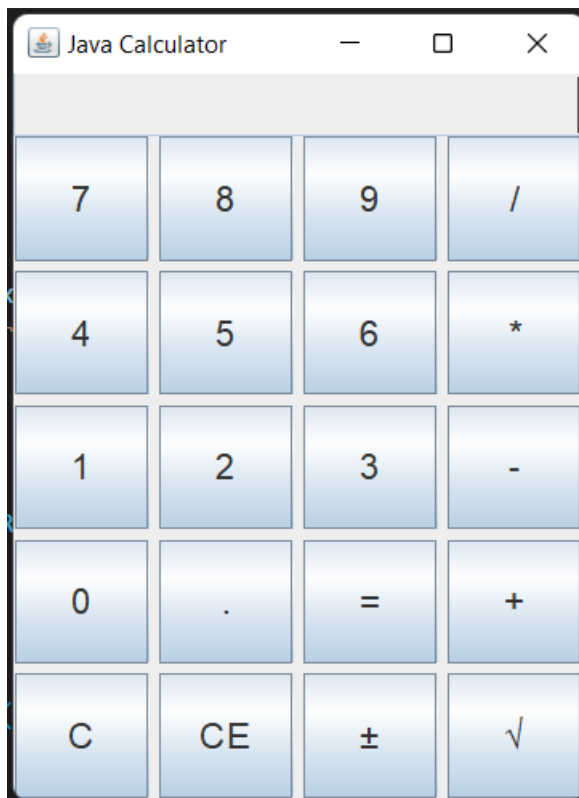
```
public static void main(String[] args) {
```

```
    SwingUtilities.invokeLater(() -> new Calculator().setVisible(true));
```

```
}
```

```
}
```

Output:



Code:

```
import javax.swing.*;

import java.awt.*;

public class BinDecConverter {

    public static void main(String[] args) {

        JTextField input = new JTextField(15), output = new JTextField(15);

        output.setEditable(false);

        JRadioButton binToDec = new JRadioButton("Binary to Decimal", true);
        JRadioButton decToBin = new JRadioButton("Decimal to Binary");

        new ButtonGroup() {

            {

                add(binToDec);

                add(decToBin);

            }

        };

        JFrame frame = new JFrame("Bin/Dec Converter");

        frame.setLayout(new GridLayout(4, 1));

        frame.add(new JPanel() {

            {

                add(new JLabel("Input:"));

                add(input);

            }

        });

        frame.add(new JPanel() {

            {
```

```

        add(binToDec);
        add(decToBin);
    }
});
frame.add(new JPanel() {
    {
        add(new JLabel("Output:"));
        add(output);
    }
});
frame.add(new JPanel() {
    {
        add(new JButton("Convert") {
            {
                addActionListener(e -> {
                    try {
                        String text = input.getText();
                        if (text.isEmpty())
                            throw new Exception("Please enter a value!");
                        output.setText(binToDec.isSelected()
binary!)
                            : (text.matches("[01]+") ? Integer.parseInt(text, 2) + "" : "Invalid
                            : (text.matches("\\d+") ? Integer.toBinaryString(Integer.parseInt(text))
                            : "Invalid decimal!"));
                    } catch (Exception ex) {
                        JOptionPane.showMessageDialog(this, ex.getMessage(), "Error", 0);
                    }
                });
            }
        });
    }
});

```

```

        }
    });
    add(new JButton("Clear") {
        {
            addActionListener(e -> {
                input.setText("");
                output.setText("");
            });
        }
    });
}

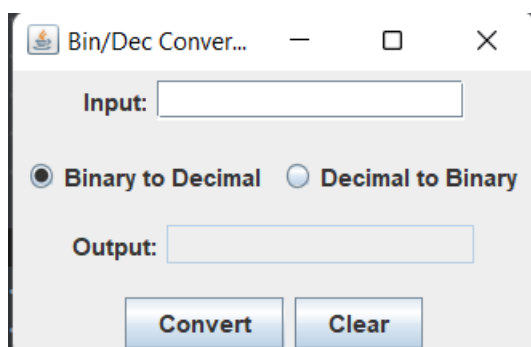
frame.pack();

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

frame.setVisible(true);
}
}

```

Output:



Code:

```
import javax.swing.*;

import java.awt.*;

public class TTT extends JFrame {

    JButton[] b = new JButton[9];

    boolean x = true;

    public TTT() {
        super("Tic-Tac-Toe");
        setSize(300, 300);
        setDefaultCloseOperation(3);
        setLayout(new GridLayout(3, 3));
        for (int i = 0; i < 9; i++) {
            b[i] = new JButton();
            b[i].setFont(new Font("Arial", 1, 50));
            b[i].addActionListener(e -> {
                JButton btn = (JButton) e.getSource();
                if (btn.getText().isEmpty()) {
                    btn.setText(x ? "X" : "O");
                    String s = x ? "X" : "O";
                    if ((b[0].getText().equals(s) && b[1].getText().equals(s) &&
b[2].getText().equals(s)) ||
                        (b[3].getText().equals(s) && b[4].getText().equals(s) &&
b[5].getText().equals(s)) ||
                        (b[6].getText().equals(s) && b[7].getText().equals(s) &&
b[8].getText().equals(s)) ||
                        (b[0].getText().equals(s) && b[3].getText().equals(s) &&
b[6].getText().equals(s)) ||
```

```

        (b[1].getText().equals(s) && b[4].getText().equals(s) &&
b[7].getText().equals(s)) ||

        (b[2].getText().equals(s) && b[5].getText().equals(s) &&
b[8].getText().equals(s)) ||

        (b[0].getText().equals(s) && b[4].getText().equals(s) &&
b[8].getText().equals(s)) ||

        (b[2].getText().equals(s) && b[4].getText().equals(s) &&
b[6].getText().equals(s))) {

            JOptionPane.showMessageDialog(this, s + " wins!");

            reset();

        } else if (!b[0].getText().isEmpty() && !b[1].getText().isEmpty() &&
!b[2].getText().isEmpty() &&

                !b[3].getText().isEmpty() && !b[4].getText().isEmpty() &&
!b[5].getText().isEmpty() &&

                !b[6].getText().isEmpty() && !b[7].getText().isEmpty() &&
!b[8].getText().isEmpty()) {

            JOptionPane.showMessageDialog(this, "Draw!");

            reset();

        } else

            x = !x;

    }

});

add(b[i]);

}

setVisible(true);

}

void reset() {

    for (JButton btn : b)

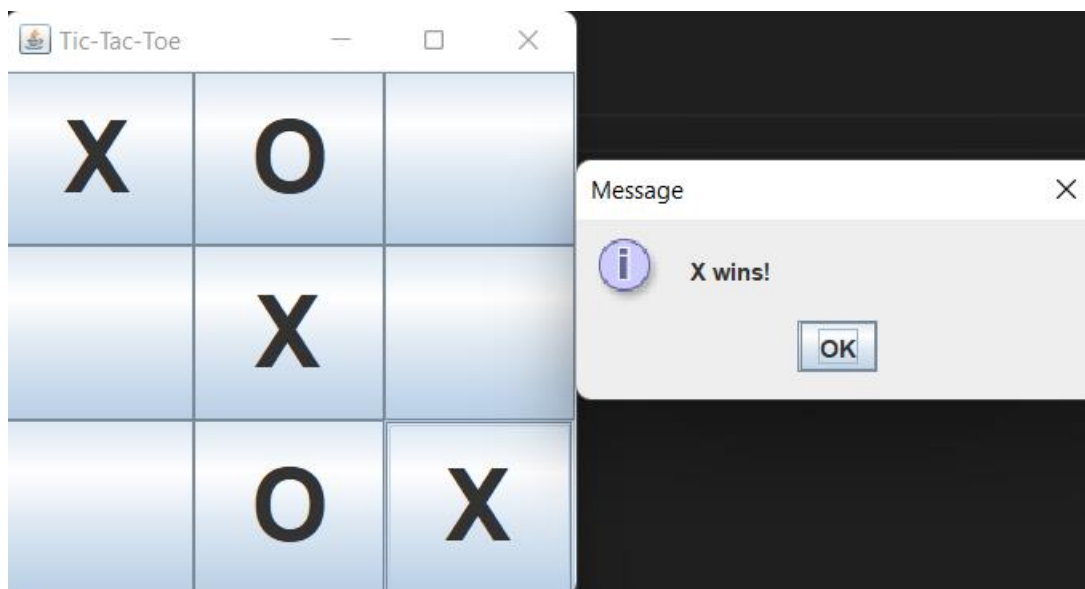
        btn.setText("");

```

```
x = true;  
}
```

```
public static void main(String[] a) {  
    new TTT();  
}  
}
```

Ouput:



Code:

```
import javax.swing.*;

public class T extends JFrame {

    JButton a = new JButton("Start"), b = new JButton("Suspend"), c = new JButton("Resume"),
    d = new JButton("Stop");

    JTextArea t = new JTextArea();

    CounterThread z; // Changed from Thread to CounterThread

    class CounterThread extends Thread {

        int k;

        boolean m, n;

        public void run() {
            while (!n)
                try {
                    synchronized (this) {
                        while (m && !n)
                            wait();
                        if (n)
                            break;
                    }
                    t.append("Count: " + (++k) + "\n");
                    sleep(500);
                } catch (Exception x) {
                    t.append("Error\n");
                }
        }
    }
}
```

```
synchronized void p() {  
    m = true;  
}
```

```
synchronized void r() {  
    m = false;  
    notify();  
}
```

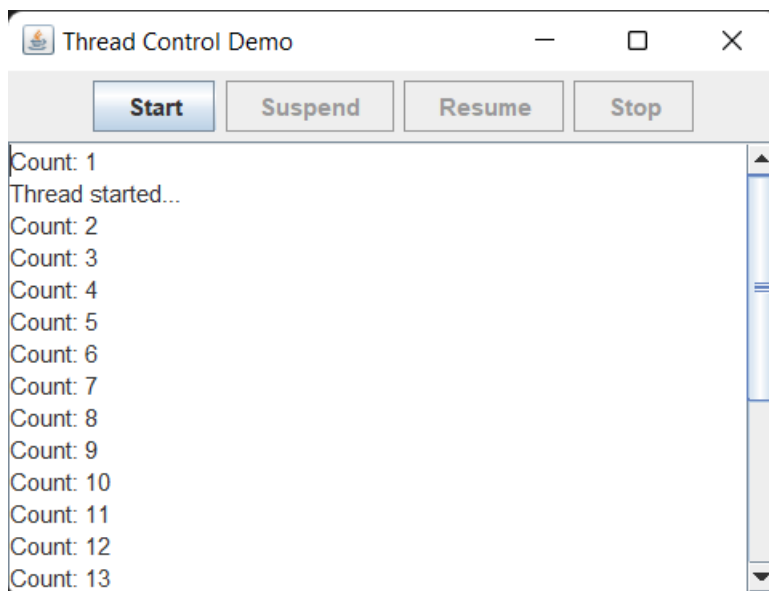
```
synchronized void s() {  
    n = true;  
    m = false;  
    notify();  
}  
}
```

```
public T() {  
    super("Thread Ctrl");  
    setSize(400, 300);  
    setDefaultCloseOperation(3);  
    JPanel p = new JPanel();  
    p.add(a);  
    p.add(b);  
    p.add(c);  
    p.add(d);  
    add(p, "North");  
    add(new JScrollPane(t));  
}
```

```
b.setEnabled(false);
c.setEnabled(false);
d.setEnabled(false);
a.addActionListener(e -> {
    z = new CounterThread();
    z.start();
    a.setEnabled(false);
    b.setEnabled(true);
    d.setEnabled(true);
    t.append("Started\n");
});
b.addActionListener(e -> {
    z.p();
    b.setEnabled(false);
    c.setEnabled(true);
    t.append("Suspended\n");
});
c.addActionListener(e -> {
    z.r();
    c.setEnabled(false);
    b.setEnabled(true);
    t.append("Resumed\n");
});
d.addActionListener(e -> {
    z.s();
    a.setEnabled(true);
    b.setEnabled(false);
    c.setEnabled(false);
```

```
        d.setEnabled(false);  
        t.append("Stopped\n");  
    });  
    setVisible(true);  
}  
  
public static void main(String[] x) {  
    new T();  
}  
}
```

Output:



Code:

```
class MyRunnable implements Runnable {  
    @Override  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(Thread.currentThread().getName() + ": " + i);  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println("Thread interrupted");  
            }  
        }  
    }  
}  
  
public class RunnableDemo {  
    public static void main(String[] args) {  
        Thread thread1 = new Thread(new MyRunnable(), "Thread-A");  
        Thread thread2 = new Thread(new MyRunnable(), "Thread-B");  
  
        thread1.start();  
        thread2.start();  
  
        System.out.println("Main thread continues execution");  
    }  
}
```


Output:

Main thread continues execution

Thread-A: 1

Thread-B: 1

Thread-A: 2

Thread-B: 2

Thread-B: 3

Thread-A: 3

Thread-A: 4

Thread-B: 4

Thread-A: 5

Thread-B: 5

Code:

```
public class T {  
  
    public static void main(String[] a) throws Exception {  
  
        new Thread(() -> t("YieldThread", 1, 0)).start();  
        new Thread(() -> t("SleepThread", 0, 1)).start();  
  
        Thread s = new Thread(() -> t("StopThread", 0, 0));  
  
        s.start();  
  
        s.join(1500);  
  
        System.out.println("\nDone");  
    }  
}
```

```
static void t(String n, int y, int s) {  
  
    for (int i = 1; i <= 5; i++) {  
  
        System.out.println(n + " " + i);  
  
        if (y > 0) {  
  
            Thread.yield();  
  
            System.out.println(n + " yielded");  
        }  
  
        if (s > 0)  
  
            try {  
  
                Thread.sleep(1000);  
  
            } catch (Exception e) {  
  
            }  
  
        if (n.equals("StopThread") && i == 3) {  
  
            System.out.println(n + " stopped");  
  
            return;  
        }  
    }  
}
```

```
    }  
    System.out.println(n + " finished");  
}  
}
```

Output:

SleepThread executing 1
StopThread executing 1
StopThread executing 2
StopThread executing 3
YieldThread executing 1
YieldThread yielded
YieldThread executing 2
StopThread stopped
YieldThread yielded
All threads completed
YieldThread executing 3
YieldThread yielded
YieldThread executing 4
YieldThread yielded
YieldThread executing 5
YieldThread yielded
YieldThread finished
SleepThread executing 2
SleepThread executing 3
SleepThread executing 4
SleepThread executing 5
SleepThread finished

Code:

Server:

```
import java.net.*;

import java.io.*;

public class S {

    public static void main(String[] a) throws Exception {

        try (ServerSocket s = new ServerSocket(12345);

            Socket c = s.accept();

            PrintWriter o = new PrintWriter(c.getOutputStream(), true);

            BufferedReader i = new BufferedReader(new
InputStreamReader(c.getInputStream())) {

                System.out.println("Connected:" + c.getInetAddress());

                o.println("Hello! Send 'exit'");

                String l;

                while ((l = i.readLine()) != null && !l.equalsIgnoreCase("exit")) {

                    System.out.println("Client:" + l);

                    o.println("Echo:" + l);

                }

                o.println("Bye!");

            }

        }

    }

}
```

Client:

```
import java.net.*;
```

```

import java.io.*;

import java.util.*;

public class C {

    public static void main(String[] a) throws Exception {

        try (Socket s = new Socket("localhost", 12345);

            PrintWriter o = new PrintWriter(s.getOutputStream(), true);

            BufferedReader i = new BufferedReader(new
InputStreamReader(s.getInputStream()));

            Scanner sc = new Scanner(System.in)) {

                System.out.println("Server: " + i.readLine());

                String l;

                while (!"exit".equalsIgnoreCase(l = sc.nextLine())) {

                    o.println(l);

                    System.out.println("Server: " + i.readLine());

                }

            }

        }

    }
}

```

Output:

Server:

Client connected: /127.0.0.1

Client: Hello server

Client:

Server: Hello from server! Send 'exit' to close.

Hello server

Server: Server received: Hello server

Code:

```
import javax.swing.*;

import java.awt.*;

import java.sql.*;

public class StudentApp extends JFrame {

    JTextField name = new JTextField(20), roll = new JTextField(20);

    JTextArea output = new JTextArea(10, 30);

    public StudentApp() {

        super("Student DB");

        setLayout(new BorderLayout());

        // Setup UI

        JPanel inputPanel = new JPanel(new GridLayout(2, 2));

        inputPanel.add(new JLabel("Name:"));

        inputPanel.add(name);

        inputPanel.add(new JLabel("Roll:"));

        inputPanel.add(roll);

        JPanel buttons = new JPanel();

        String[] btnLabels = { "Insert", "Edit", "Delete", "Show" };

        for (String label : btnLabels) {

            JButton btn = new JButton(label);

            btn.addActionListener(e -> handleAction(label));

            buttons.add(btn);

        }

    }

}
```

```
output.setEditable(false);  
add(inputPanel, BorderLayout.NORTH);  
add(buttons, BorderLayout.CENTER);  
add(new JScrollPane(output), BorderLayout.SOUTH);
```

```
setDefaultCloseOperation(EXIT_ON_CLOSE);  
pack();  
setVisible(true);  
}
```

```
private void handleAction(String action) {  
    try {  
        switch (action) {  
            case "Insert":  
                execute("INSERT INTO students VALUES (?,?)", name.getText(), roll.getText());  
                break;  
            case "Edit":  
                execute("UPDATE students SET name=? WHERE roll_number=?", name.getText(),  
roll.getText());  
                break;  
            case "Delete":  
                execute("DELETE FROM students WHERE roll_number=?", roll.getText());  
                break;  
            case "Show":  
                showStudents();  
                break;  
        }  
    }  
}
```

```

    } catch (Exception e) {
        output.setText("Error: " + e.getMessage());
    }
}

```

```

private void execute(String sql, String... params) throws SQLException {
    try (Connection c =
DriverManager.getConnection("jdbc:mysql://localhost:3306/student_db", "root", "")) {
        PreparedStatement ps = c.prepareStatement(sql) {
            for (int i = 0; i < params.length; i++)
                ps.setString(i + 1, params[i]);
            output.setText(ps.executeUpdate() > 0 ? "Success" : "Failed");
            name.setText("");
            roll.setText("");
        }
    }
}

```

```

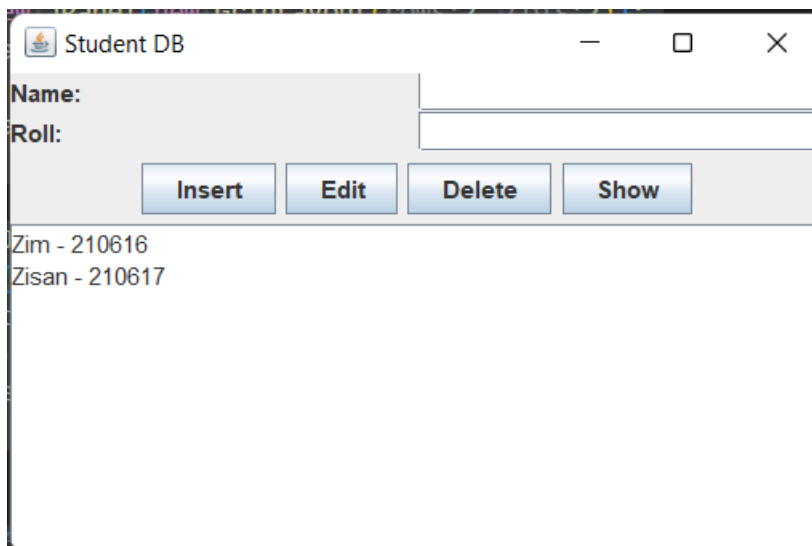
private void showStudents() throws SQLException {
    try (Connection c =
DriverManager.getConnection("jdbc:mysql://localhost:3306/student_db", "root", "")) {
        Statement st = c.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM students") {
            StringBuilder sb = new StringBuilder();
            while (rs.next())
                sb.append(rs.getString(1)).append(" - ").append(rs.getString(2)).append("\n");
            output.setText(sb.length() > 0 ? sb.toString() : "No data");
        }
    }
}

```



```
public static void main(String[] args) {  
    new StudentApp();  
}  
}
```

Output:



Student DB

Name:

Roll:

Zim - 210616
Zisan - 210617